



NAZARBAYEV
UNIVERSITY



Report of Final Project of ROBT 501

Almat Dalabekov
Bauyrzhan Askarov
Dias Akimbay

Introduction

The Niryo-One is a 6-DOF RRRRRR manipulator, making it fully capable of arbitrary end-effector positioning and orientation. According to the project requirements, the entire kinematic modeling must be performed using screw displacements rather than DH parameters. The goals are:

1. Symbolically derive the forward kinematics (FK) using PoE.
2. Derive a symbolic closed-form inverse kinematics (IK) solution.
3. Validate both FK and IK using CoppeliaSim and MATLAB Remote API.

All modeling is conducted with the robot at its zero configuration, where all joints are set to 0° . In order to find the forward and inverse kinematics of the manipulator, we defined the positions of each screw axis and the base frame as can be seen below:

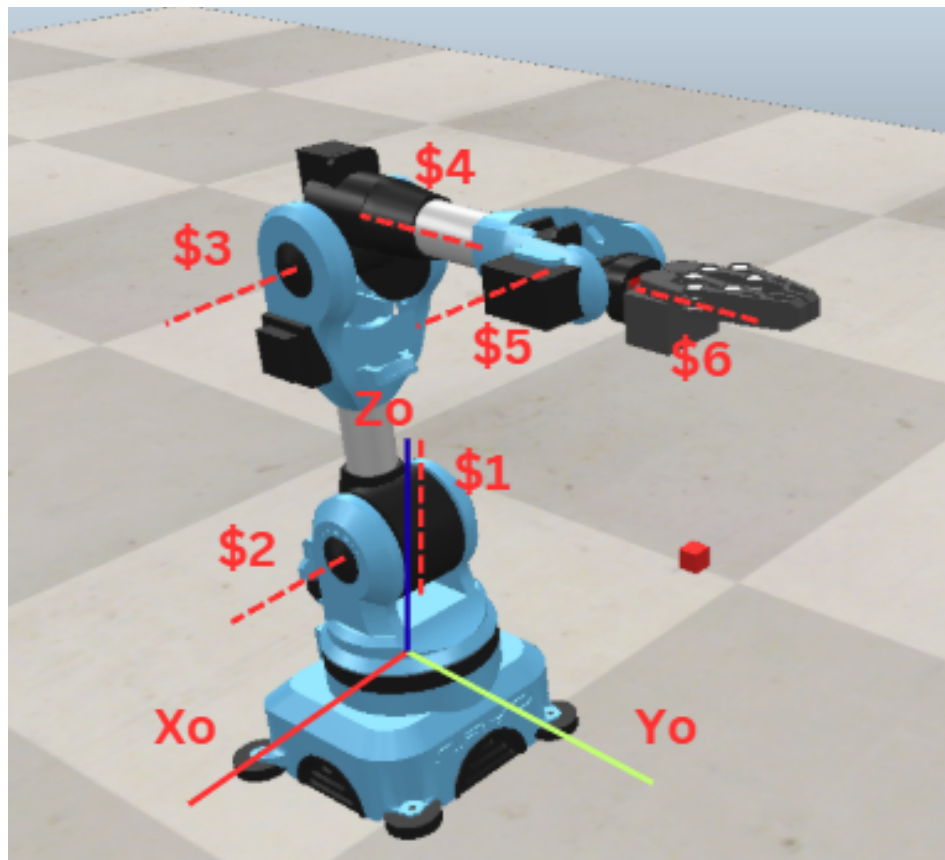


Figure 1. Screw-axis assignment for each joint of the manipulator

Kinematic Structure and Screw Axes

Based on the screw-axis configuration shown in Figure 1, the corresponding screw-axis table for the robot model was derived and subsequently used for the analysis of the robot's motion:

Joint i	Screw Axis S_i	Moment S_{0i}
1	$(0, 0, 1)$	$(0, 0, 0)$
2	$(1, 0, 0)$	$(0, 0, L_1)$
3	$(1, 0, 0)$	$(0, 0, L_1 + L_2)$
4	$(0, 1, 0)$	$(0, 0, L_1 + L_2 + \text{offset})$
5	$(1, 0, 0)$	$(0, L_3, L_1 + L_2 + \text{offset})$
6	$(0, 1, 0)$	$(0, 0, L_1 + L_2 + \text{offset})$

Note that we picked a point of origin of screw axes 4 and 6 in the position where it intersects with the base frame's z axis to simplify our model later for inverse. The constants such as L_1 , L_2 , L_3 are the lengths of the manipulator and offset depicted in the figure below:

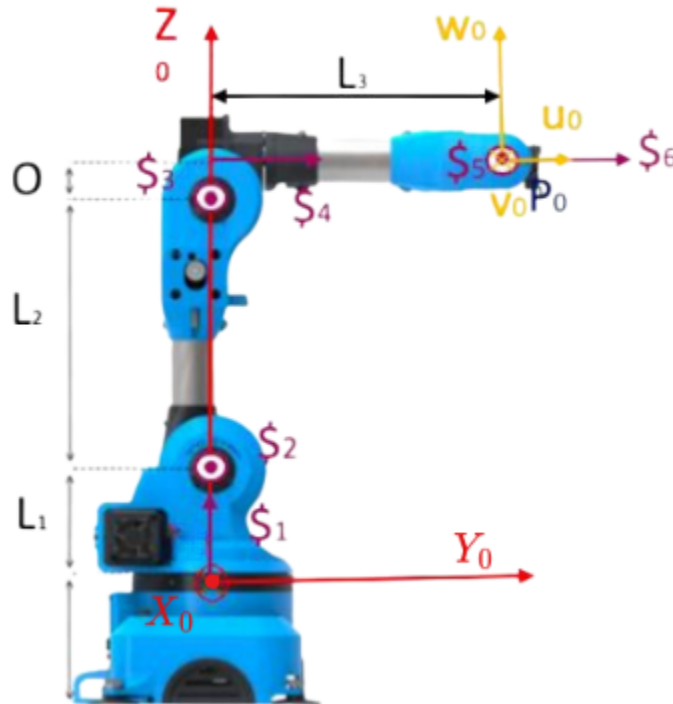


Figure 2. Manipulator's schematic diagram

Product of Exponentials Formulation

Each joint contributes a screw transformation. For revolute joints:

$$A_i = \begin{bmatrix} R_i & (I - R_i) S_{0i} \\ 0 & 1 \end{bmatrix}$$

We derived all six matrices using the screw transformation and the rodriques formulas for: A_1 (rotation about Z), A_2 , A_3 , A_5 (rotations about X), A_4 , A_6 (rotations about Y). Therefore the final result for our matrices are shown below:

$$A_1 = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_2 & -\sin \theta_2 & k_2 \sin \theta_2 \\ 0 & \sin \theta_2 & \cos \theta_2 & k_2(1 - \cos \theta_2) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_3 & -\sin \theta_3 & k_3 \sin \theta_3 \\ 0 & \sin \theta_3 & \cos \theta_3 & k_3(1 - \cos \theta_3) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_4 = \begin{bmatrix} \cos \theta_4 & 0 & \sin \theta_4 & -k_4 \sin \theta_4 \\ 0 & 1 & 0 & 0 \\ -\sin \theta_4 & 0 & \cos \theta_4 & k_4(1 - \cos \theta_4) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_5 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_5 & -\sin \theta_5 & k_5(1 - \cos \theta_5) + k_4 \sin \theta_5 \\ 0 & \sin \theta_5 & \cos \theta_5 & -k_5 \sin \theta_5 + k_4(1 - \cos \theta_5) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_6 = \begin{bmatrix} \cos \theta_6 & 0 & \sin \theta_6 & -k_4 \sin \theta_6 \\ 0 & 1 & 0 & 0 \\ -\sin \theta_6 & 0 & \cos \theta_6 & k_4(1 - \cos \theta_6) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where the constants $k_2 = L_1$, $k_3 = L_1 + L_2$, $k_4 = L_1 + L_2 + O$, $k_5 = L_3$

Forward Kinematics

Now we have all six matrices to solve for forward kinematics of the manipulator. The final Matrix for the wrist frame is calculated as follows:

$$A_n = A_1 A_2 A_3 A_4 A_5 A_6$$

However to valide it in the simulation environment, the position and orientation of the end effector was chosen to demonstrate the produced FK of the Niryo-One manipulator. In this sense, some adjustments for our final matrix should be made. As the base frame of the manipulator is not specified in a global frame we used translational matrix A_0 in order to map the global position metrics for validation. Moreover, as the final goal is the position of the end effector, the translation part after the wrist should also be made by using the matrix A_{ee} . So the total FK transform:

$$T_{\text{final}} = A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_{ee}$$

Where A_0 can be adjusted depending on where robot is placed in the simulation, and the translational matrix A_{ee} for end-effector is derived as this from the CoppeliaSim:

$$A_{ee} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0.27 \\ 0 & 0 & 1 & 0.357 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

We tested our Forward Kinematics in these configurations:

- 1) When $\theta_1 = 70$ degrees
- 2) When $\theta_2 = 20$ degrees
- 3) When $\theta_2 = -30$ degrees, $\theta_5 = 40$ degrees
- 4) When $\theta_1 = 45$, $\theta_2 = 15$, $\theta_3 = 10$, $\theta_4 = -20$, $\theta_5 = 10$, $\theta_6 = 20$ degrees

Validation in CoppeliaSim is shown on the next page

x: -0.92753 y: +0.54264 z: +0.40882
a: +90.00 b: -20.00 g: 0.00

Ttotal =

0.3420	-0.9397	0	-0.9287
0.9397	0.3420	0	0.5363
0	0	1.0000	0.4080
0	0	0	1.0000

1)

Position: x: -0.6743 y: +0.62564 z: +0.48711
Orientation: a: 0.00 b: -90.00 g: -110.00

Ttotal =

1.0000	0	0	-0.6750
0	0.9397	-0.3420	0.6208
0	0.3420	0.9397	0.4868
0	0	0	1.0000

2)

x: -0.6743 y: +0.81113 z: +0.27878
a: 0.00 b: -90.00 g: -105.00

Ttotal =

1.0000	0	0	-0.6750
0	0.9659	-0.2588	0.8055
0	0.2588	0.9659	0.2704
0	0	0	1.0000

3)

Shape (cuboid)

x: -0.80441 y: +0.57633 z: +0.52157
a: +92.277 b: -42.009 g: -32.832

Ttotal =

0.6692	-0.6244	0.4029	-0.8060
0.7425	0.5404	-0.3959	0.5717
0.0295	0.5641	0.8252	0.5202
0	0	0	1.0000

4)

Inverse Kinematics - wrist position independence

Let the desired end-effector pose be

$$P_{\text{des}} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}.$$

The forward kinematics of the manipulator can be expressed as

$$P_{\text{des}} = A_1 A_2 A_3 A_4 A_5 A_6 p_0,$$

A_i are the spatial screw transforms, and P_0 is the wrist point in home configuration. Each screw transform has the form:

$$A_i = \begin{bmatrix} R_i & d_i \\ 0 & 1 \end{bmatrix}.$$

For a revolute joint, the translational component is

$$d_i(\theta_i) = (I - R_i) c_i + q_i,$$

Where c_i is the screw origin. q_i is the moment vector.

$$P_0 = [0, L_3, L_1 + L_2 + \text{offset}] = [0, k_5, k_4]$$

Conclusion: Wrist rotation does not move the Wrist Point!

The combined transform $A_4 A_5 A_6$ of the wrist joints. Since joints 4-6 intersect at the wrist center, all the screw axes share the same origin. But in computing the forward

kinematics part, their origin could have been the same, but treating y component as 0 instead, L3 was cost-efficient for FK computation.

Thus:

$$d_4 = d_5 = d_6 = q.$$

$$A_4 A_5 A_6 p_0 = R_4 R_5 R_6 p_0 + R_4 R_5 d_6 + R_4 d_5 + d_4.$$

$$R_4 R_5 (q - R_6 q) + R_4 (q - R_5 q) + (q - R_4 q).$$

$$q(R_4 R_5 - R_4 R_5 R_6) + q(R_4 - R_4 R_5) + q(I - R_4).$$

$$= q[I - R_6 R_5 R_4].$$

So for a Wrist intersection, p_0 is equal to q . This means:

$$A_4 A_5 A_6 p_0 = p_0.$$

Therefore:

$$P_{\text{des}} = A_1 A_2 A_3 p_0.$$

Wrist-Point Extraction From Desired End-Effector Pose

The desired end-effector pose is:

$$T_{\text{ee,des}}.$$

We need to compute the wrist-point position p_x , p_y , p_z . We have in the home position p_{wrist_0} , and p_{ee_0} . In MATLAB notation, we wrote:

$$\begin{aligned}
k_5 &= L_3, \\
k_4 &= L_4 + L_5 + \text{off}, \\
k_3 &= L_2 + L_3, \\
k_2 &= L_1.
\end{aligned}$$

And here is the Relationship between the end-effector frame and the Wrist frame.

$$T_{\text{ee,des}} = A_{1 \rightarrow 6} T_{\text{end-off},0}.$$

$$T_{\text{wrist,des}} = A_{1 \rightarrow 6} T_{\text{wrist},0}.$$

$$T_{\text{ee,des}} = T_{\text{wrist,des}} T_{\text{off}},$$

$$T_{\text{wrist,des}} = T_{\text{ee,des}} T_{\text{end-off},0}^{-1}.$$

$$T_{\text{wrist,des}} = T_{\text{ee,des}} \begin{bmatrix} I & -t_{\text{end}} \\ 0 & 1 \end{bmatrix},$$

$$T_{\text{wrist,des}} = \begin{bmatrix} R_{\text{wrist}} & t_{\text{wrist}} \\ 0 & 1 \end{bmatrix}.$$

$$\begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}_{\text{given}} = R_{\text{ee,des}} (t_{\text{wrist},0} - t_{\text{ee},0}) + t_{\text{ee,des}}.$$

$$(t_{\text{wrist},0} - t_{\text{ee},0}) = [0, -k_6, -k_7]$$

So p_x, p_y, p_z is:

$$p = R_{ee,des} \begin{bmatrix} 0 \\ -k_6 \\ -k_7 \end{bmatrix} + t_{ee,des}$$

Solving for θ_1 .

Start from the transformed position

$$\tilde{p} = A_1^{-1} p_{des} = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \end{bmatrix} = A_2 A_3 p_0. \quad \begin{bmatrix} 0 \\ k_2 s_2 - k_3 s_2 + k_3 s_{23} - k_4 s_{23} + k_5 c_{23} \\ -k_2 c_2 + k_2 + k_3 c_2 - k_3 c_{23} + k_4 c_{23} + k_5 s_{23} \\ 1 \end{bmatrix}$$

The rotation about joint 1 is a rotation around the base z-axis. Writing the effect of a rotation by θ_1 on the x, y coordinates gives the pair of scalar equations

$$0 = c_1 p_x + s_1 p_y,$$

$$r_2 = -s_1 p_x + c_1 p_y,$$

$$r_3 = p_z.$$

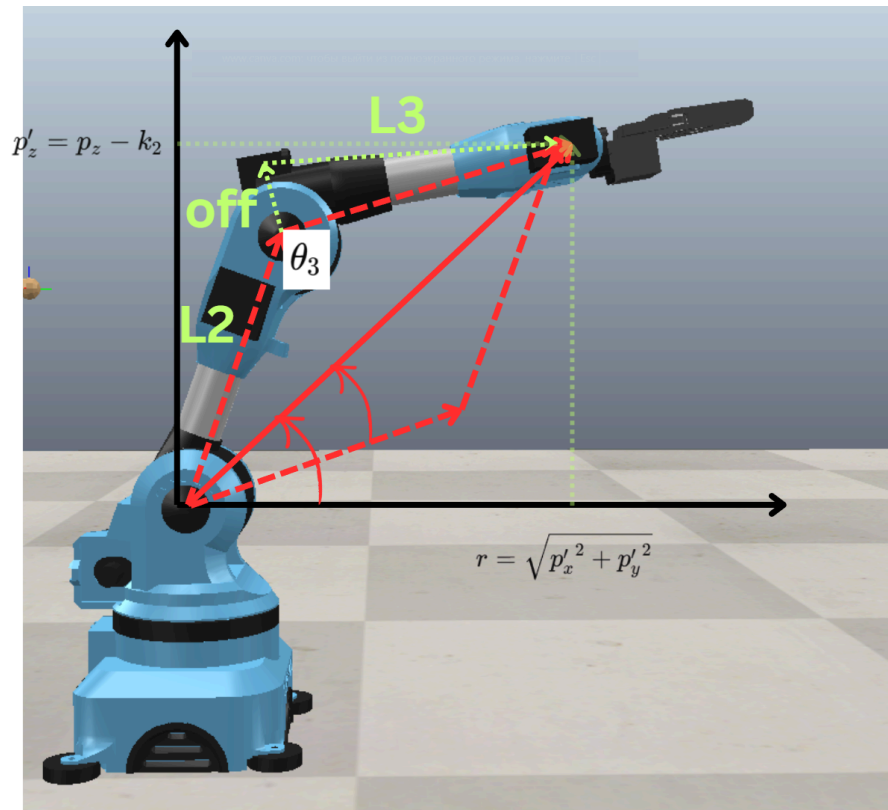
$$p_x \cos \theta_1 + p_y \sin \theta_1 = 0 \quad \implies \quad \frac{\sin \theta_1}{\cos \theta_1} = -\frac{p_x}{p_y}.$$

A numerically stable way to compute θ_1 that handles quadrant correctly is to use the two-argument arctangent:

$$\theta_1 = \text{atan2}(-p_x, p_y). \quad \theta_1^{(2)} = \theta_1 + \pi.$$

Because tangent has period π , there is a second equivalent solution.

Solving for θ_3 .



Rotate the desired wrist-centre position into the plane of joints 2–3 (i.e. after removing θ_1 and call the coordinates.

$$p'_x, p'_y, p'_z.$$

$$r = \sqrt{p_x'^2 + p_y'^2}$$

The r is the planar distance from Joint2 to the wrist center

Often p_z' is written as $p_z' = p_z - k_2$ if there is an offset k_2 between frames — use whatever constant offsets your robot has.

$$\text{hyp} = \sqrt{p_x'^2 + p_y'^2 + p_z'^2}.$$

$$D = \frac{p_x'^2 + p_y'^2 + p_z'^2 - \text{off}^2 - L_2^2 - L_3^2}{2 L_2 \text{hyp}}.$$

Check reachability if $|D| > 1$, there is no real solution.

$$\sin \theta_3 = \pm \sqrt{1 - D^2}, \quad \theta_3 = \text{atan2}(\pm \sqrt{1 - D^2}, D),$$

Which gives the two elbow configurations.

Solving for θ_2 .

Look at the triangle from joint 2 to the wrist center (hypotenuse) and project it into the plane containing joints 2 and 3.

$$\alpha = \text{atan2}(p_z', r).$$

From triangle geometry, the internal angle contributed by the two-link elbow at joint 3 is

$$\beta = \text{atan2}(L_3 \sin \theta_3, L_2 + L_3 \cos \theta_3).$$

By using two θ_3 , in code, we solve for different two θ_2 solutions.

$$\theta_2^{(1)} = \text{atan2}(p_z', r) - \text{atan2}(L_3 \sin \theta_3^{(1)}, L_2 + L_3 \cos \theta_3^{(1)}),$$

$$\theta_2^{(2)} = \text{atan2}(p_z', r) - \text{atan2}(L_3 \sin \theta_3^{(2)}, L_2 + L_3 \cos \theta_3^{(2)}).$$

L_3 here is not the real L_3 constant. Rather $= \sqrt{L_3^2 + \text{offset}^2}$.

Solving for θ_4 , θ_5 , θ_6 .

Once we have the first three joints, we can isolate the wrist orientation.

$$T_{\text{des}} = A_{1 \rightarrow 3} A_{4 \rightarrow 6} T_{\text{end},0}$$

$$A_{4 \rightarrow 6} T_{\text{end},0} = (A_{1 \rightarrow 3})^{-1} T_{\text{ee,des}}.$$

$$A_{4 \rightarrow 6} T_{\text{end},0} = (A_{1 \rightarrow 3})^{-1} T_{\text{ee,des}}, \quad T_{\text{ee,des}} = \begin{bmatrix} h_x & o_x & a_x & n_x \\ h_y & o_y & a_y & n_y \\ h_z & o_z & a_z & n_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Applying the Translational removal:

$$A_{4 \rightarrow 6} T_{\text{end},0} = \begin{bmatrix} R_{4 \rightarrow 6} & d_{4 \rightarrow 6} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} I & t_{\text{end}} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} R_{4 \rightarrow 6} & R_{4 \rightarrow 6} t_{\text{end}} + d_{4 \rightarrow 6} \\ 0 & 1 \end{bmatrix}.$$

$$R_{456} = \begin{bmatrix} c_4 c_6 - s_4 c_5 s_6 & -s_4 s_5 & c_4 s_6 + s_4 c_5 c_6 \\ s_5 s_6 & c_5 & -s_5 c_6 \\ -c_4 c_5 s_6 - s_4 c_6 & c_4 s_5 & -c_4 c_5 c_6 + s_4 s_6 \end{bmatrix}$$

From the third row of the rotation matrix:

$$\theta_5 = \text{atan2}\left(\pm \sqrt{o_x^2 + o_y^2}, o_y\right) \quad (\text{two solutions})$$

$$\theta_4 = \pi - \text{atan2}\left(-\frac{o_x}{\sin \theta_5}, \frac{o_y}{\sin \theta_5}\right)$$

$$\theta_6 = \text{atan2}\left(\frac{h_y}{\sin \theta_5}, \frac{a_y}{\sin \theta_5}\right)$$

Discussion of θ_4 , θ_5 , θ_6

During the inverse kinematics (IK) computation of the wrist joints (θ_4 , θ_5 , θ_6), the mathematical derivation typically produces an expression of the form:

$$\theta_4 = -\text{atan2}\left(\frac{o_x}{\sin \theta_5}, \frac{o_y}{\sin \theta_5}\right)$$

However, in implementation, it is often necessary to use the modified expression

$$\theta_4 = \pi - \text{atan2}\left(-\frac{o_x}{\sin \theta_5}, \frac{o_y}{\sin \theta_5}\right)$$

In order for the inverse kinematics solver to match the correct robot orientation.

Although this π term does not explicitly appear in the purely symbolic derivation, there are several valid reasons why it becomes required in practice.

The atan2 function returns angles strictly within the interval $(-\pi, \pi]$.

However, a tangent equation such as:

$$\tan \theta_4 = \frac{o_y}{o_x}$$

Has infinitely many solutions:

$$\theta_4, \theta_4 + \pi, \theta_4 + 2\pi, \theta_4 + 3\pi, \dots$$

The robot's physical configuration may correspond to a solution that differs by π from the one chosen automatically by $\text{atan2}()$.

Adding or subtracting π selects the correct “branch” of the solution.

Most 6-DOF arms use a spherical wrist (joints 4, 5, and 6 intersect at one point).

This design introduces a natural symmetry:

- If θ_5 flips sign,
- Then θ_4 and θ_6 must rotate by π to produce the same end-effector orientation.

This effect is known as the *wrist flip*. Thus, even if the mathematically derived form of θ_4 does not contain π , the physically correct orientation may require:

The rotation matrix used for joints 4–6 in your derivation has the structure

- first column begins with c_4c_5
- third row begins with $-s_5$
- second and third columns contain additional sign changes and cross-coupled terms

This matrix does not follow the classical Z-Y-Z or Z-Y-X wrist rotation convention used in textbooks. Because the axes differ in sign and direction, the angles that satisfy this matrix differ by π from those produced by the standard inverse-trigonometric formulas. Thus, the π term is not arbitrary. It compensates for different frame definitions and axis orientations. Even if the correct robot configuration requires $\theta_4 \approx 180^\circ$, the atan2 expression may return 0° or -180° because those values satisfy the same tangent relationship.

To obtain the physically correct orientation that matches your robot’s FK, a π offset is needed.

The goal of Inverse Kinematics (IK) for the Niryo One is to determine the six joint variables. In particular, $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$. Given a desired end-effector pose:

$$T_{\text{des}} = \begin{bmatrix} R_{\text{des}} & p_{\text{des}} \\ 0 & 1 \end{bmatrix}$$

Because the Niryo-One is a 6-DOF RRRRRR robot with a spherical wrist, the IK can be solved analytically by decoupling:

Position IK $\rightarrow$ determines $\theta_1, \theta_2, \theta_3$

Orientation IK $\rightarrow$ determines $\theta_4, \theta_5, \theta_6$

The screw axis structure uniquely simplifies the derivation:

Screws 2 and 3 are parallel $\rightarrow$ their combined motion behaves like a planar 2-link arm in the sagittal plane.

Screws 4–5–6 intersect at the wrist $\rightarrow$ orientation can be solved via a Y–X–Y rotation sequence.

These properties allow us to derive a clean, closed-form IK solution.

Wrist Decoupling via Screw Theory

Using the Product of Exponentials formula $T_{des} = A_1 A_2 A_3 A_4 A_5 A_6 A_{ee} T_{des}$, we isolate the wrist rotation: $A_{456} = A_4 A_5 A_6$. Because screws 4, 5, and 6 intersect at the wrist center, their product represents a pure orientation transformation. The IK strategy:

1. Solve $\theta_1, \theta_2, \theta_3$ using position constraints.
2. Compute wrist rotation
3. Extract $\theta_4, \theta_5, \theta_6$ from the rotation matrix.

This decomposition is supported by the slide showing the Y–X–Y wrist structure.

Solving θ_1 using Screw Geometry

From the derivation $A_1^{-1} T_{final} = A_2 A_3 A_4 A_5 A_6 P_0$, and from multiplying by A_1^{-1} we obtain:

$$\begin{bmatrix} 0 \\ r_2 \\ r_3 \end{bmatrix} = A_1^{-1} \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

From the first row: $0 = c_1 p_x + s_1 p_y$. Thus: $\theta_1 = \text{atan2}(-p_x, p_y)$. This aligns exactly with the geometry: joint 1 rotates around the z-axis, aligning the arm toward the projection of the target on the XY-plane.

Solving θ_2 and θ_3 using Parallel Screw Axes

Screws 2 and 3 are parallel (both rotate about the x-axis).

Therefore:

- Their composition behaves like a planar 2-link manipulator.
- Combined angles appear as s23,c23 as shown in your derivation matrix.

After rotating the target by $-\theta_1$:

$$\begin{bmatrix} p'_x \\ p'_y \\ p'_z \end{bmatrix} = R_z(-\theta_1) \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}.$$

From the geometric diagram, we define: $x=p_y'$, $z=p_z'$.

The effective link lengths are:

$$L_2 = k_3 - k_2, \quad L_3 = ((k_3 - k_4)^2 + (k_5)^2)^{1/2}$$

Where the constants are:

$$k_2 = L_1, \quad k_3 = L_1 + L_2, \quad k_4 = L_1 + L_2 + \text{offset}, \quad k_5 = L_3$$

Using law of Cosines for θ_3 , we get:

$$\theta_3^{(\pm)} = \text{atan2}\left(\pm\sqrt{1 - D^2}, D\right)$$

The $\pm$ corresponds to elbow up / elbow down configurations.

Solving θ_2

$$\theta_2 = \text{atan2}(z, x) - \text{atan2}(L_3 \sin \theta_3, L_2 + L_3 \cos \theta_3)$$

This expression arises naturally from the planar geometry shown on your slide and produces two possible IK branches.

Solving θ_4 , θ_5 , θ_6 from Wrist Orientation

Once the first three joints determine the wrist position, the remaining joints contribute only rotation.

$$A_{4 \rightarrow 6} T_{\text{end},0} = (A_{1 \rightarrow 3})^{-1} T_{\text{ee,des}}, \quad T_{\text{ee,des}} = \begin{bmatrix} h_x & o_x & a_x & n_x \\ h_y & o_y & a_y & n_y \\ h_z & o_z & a_z & n_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The wrist rotation:

$$R_{456} = (R_1 R_2 R_3)^{-1} R_{\text{des}}.$$

We computed the analytical expression of R_{456} :

$$R_{456} = \begin{bmatrix} c_4 c_6 - s_4 c_5 s_6 & -s_4 s_5 & c_4 s_6 + s_4 c_5 c_6 \\ s_5 s_6 & c_5 & -s_5 c_6 \\ -c_4 c_5 s_6 - s_4 c_6 & c_4 s_5 & -c_4 c_5 c_6 + s_4 s_6 \end{bmatrix}$$

Comparing this with the desired rotation matrix, we solved angles:

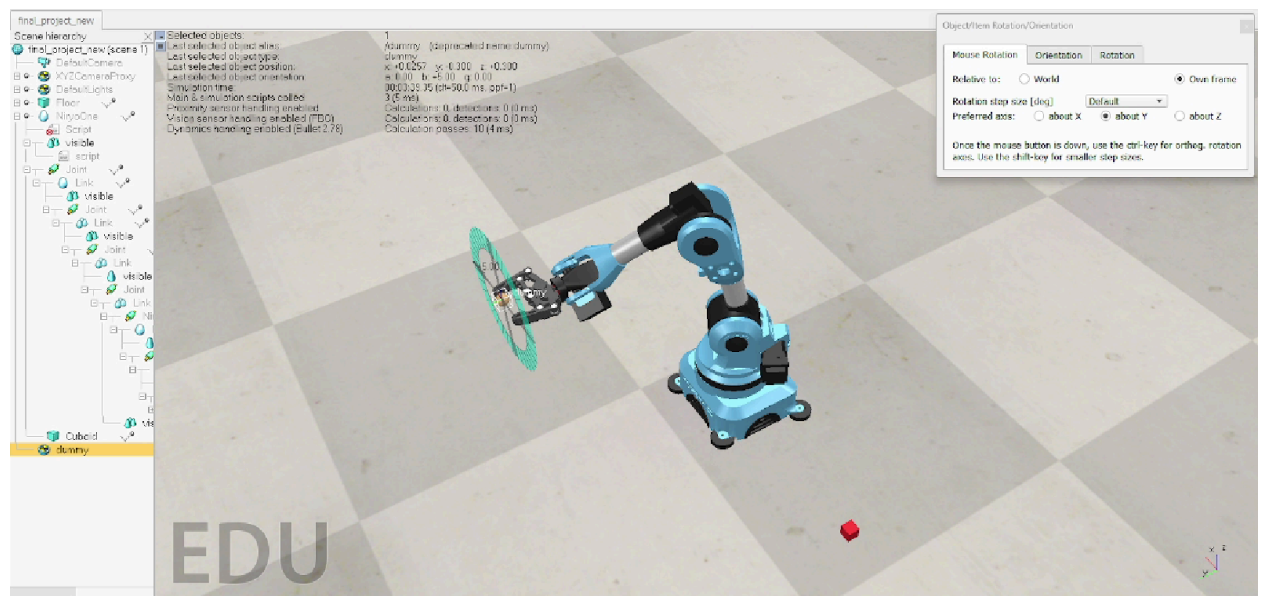
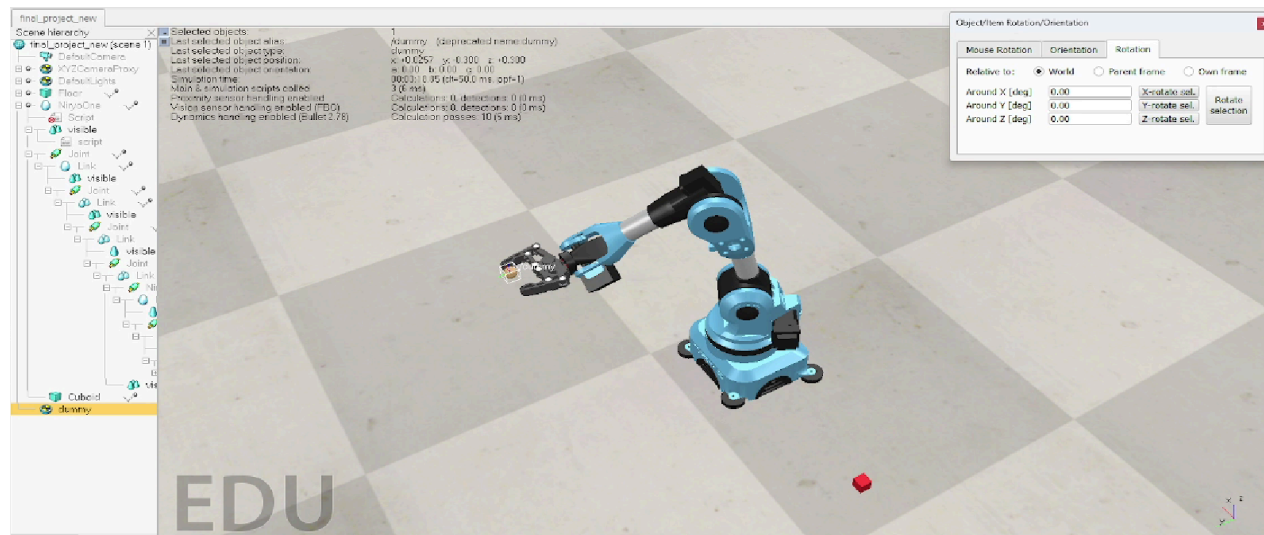
$$\theta_5 = \text{atan2} \left(\sqrt{o_x^2 + o_z^2}, o_y \right)$$

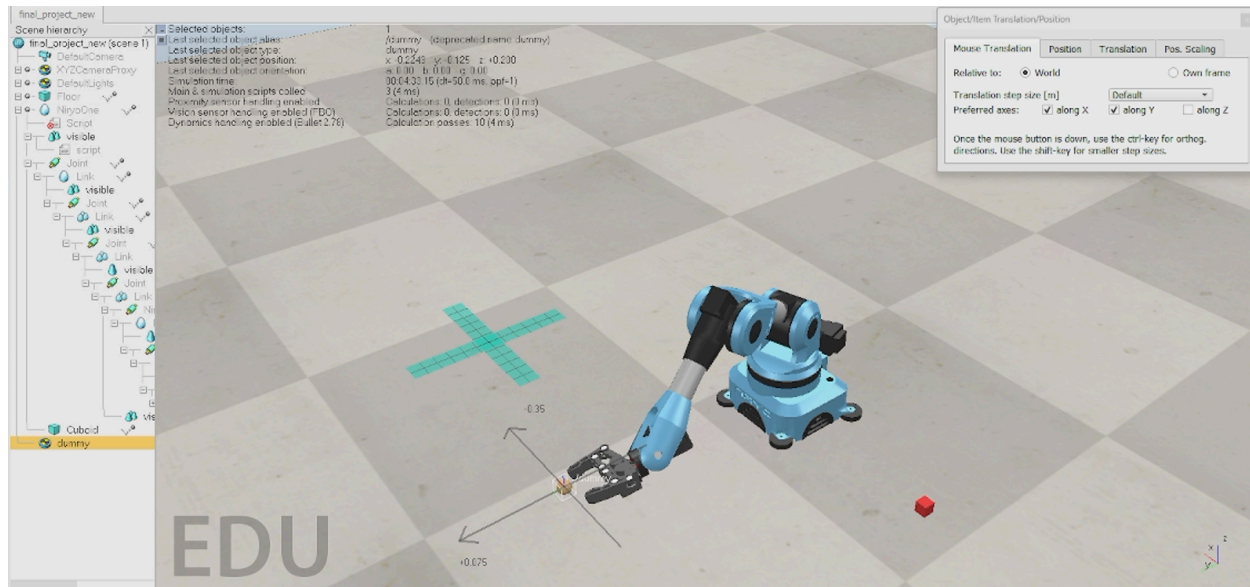
$$\theta_4 = \pi - \text{atan2} \left(-\frac{o_x}{\sin \theta_5}, \frac{o_y}{\sin \theta_5} \right)$$

$$\theta_6 = \text{atan2} \left(\frac{h_y}{\sin \theta_5}, \frac{a_y}{\sin \theta_5} \right)$$

Simulation Validation

Finally to validate the Inverse closed-loop solution, experiments were carried out. We created a simple experiment with the principle of giving the position and orientation using some dummy variable in the CoppeliaSim and running the code of inverse through the matlab code. The goal was simply to follow the position of the dummy variable. The experiment showed good results in following the dummy variable:





The video demonstration of the experiment can be accessed with this link:

https://drive.google.com/drive/folders/1EQuRee-gZgJD_2OQNUHWPYNPFswO9YEK?usp=sharing

The code explanations can be found in Appendix section after the conclusion. All the code and scene file can be found in the .zip file

Conclusion

In this project, the kinematic behavior of the 6-DOF Niryo-One manipulator was analyzed and modeled entirely through the principles of screw theory, as required by the project specification. By applying the Successive screw method formulation, we derived a complete symbolic representation of the forward kinematics, enabling an explicit mathematical description of the end-effector pose as a function of joint variables. Building on this foundation, a closed-form inverse kinematics solution was constructed. The structure of the robot, particularly the parallel nature of screws 2 and 3 and the intersecting wrist axes (4–5–6), allowed the IK problem to be decomposed into position and orientation subproblems. This yielded analytically tractable expressions for all joint angles, including the two possible elbow configurations that arise naturally from the geometry of a 2-link shoulder–elbow subsystem.

The symbolic derivations were implemented and evaluated in MATLAB, where both the forward and inverse kinematic models were executed and solved with high numerical consistency. These results were then validated using CoppeliaSim via the Remote API interface. By commanding the robot with computed joint solutions and comparing simulated end-effector poses against the analytically predicted ones, we verified the

correctness and accuracy of our models. The close agreement between the theoretical transformations and the simulator outputs demonstrates that screw-theoretic modeling provides a precise and robust method for kinematic analysis of the Niryo-One. Overall, the project successfully achieved all objectives, providing a complete mathematical and computational framework for both forward and inverse kinematics of the manipulator.

Appendix

We provided a final IK demonstration in CoppeliaSim. The code written for the Kinematics configuration is explained below:

- 1) Configuration of the CoppeliaSim to MATLAB. This code provides basic definitions of variables used throughout the simulation and connection procedure using the MATLAB API.

```
%% ----- constants / limits used by IK -----
k2 = 0.132; k3 = 0.342; k4 = 0.372;
k5 = 0.2317; k6 = 0.1033; k7 = 0.00748;

planar_link2 = k3 - k2;
planar_link3 = sqrt((k4 - k3)^2 + k5^2);

limits_deg = [ -175, 175; % theta1
               -90, 36.7; % theta2
               -80, 90; % theta3
               -175, 175; % theta4
               -100, 110; % theta5
               -147.5, 147.5 ]; % theta6
limits_rad = deg2rad(limits_deg);

% initial warm-start guess for full theta
theta_guess = deg2rad([10; 10; 10; 10; 10; 10]);

%% ----- Config -----
PORT = 19991; % change if your scene uses different port
dummyCandidates = {'dummy', '/NiryoOne/dummy', '/NiryoOne/dummy', '/NiryoOne/dummy'};
solveInterval = 0.20; % seconds between IK solves (rate-limited)
printPose = true; % print dummy pose each loop
maxSolveTime = 1.5; % seconds allowed for solver try (soft)

%% ----- connect -----
sim = remApi('remoteApi');
sim.simxFinish(-1);
clientID = sim.simxStart('127.0.0.1', PORT, true, true, 5000, 5);
if clientID < 0
    sim.delete();
    error('Cannot connect to CoppeliaSim on port %d. Start remote API');
end
fprintf('Connected to CoppeliaSim on port %d (clientID=%d)\n', PORT, clientID);
```

- 2) In the provided Scene environment file for CoppeliaSim software, we changed some names of joints and dummy variables for which code for identifying the names was used:

```

%% ----- find joint handles robustly -----
basevariants = { 'h1ry0one_joint', 'h1ry0one_joint', 'h1ry0one_joint', 'h1ry0one_joint' };
foundHandles = nan(1,6);
for i = 1:6
    found = false;
    triedNames = {};
    for v = 1:numel(basevariants)
        namesToTry = {
            sprintf('%s%d', basevariants(v), i), ...
            sprintf('%s%d', basevariants(v), i), ...
            sprintf('%s_joint%d', basevariants(v), i), ...
            sprintf('%s_joint%d', basevariants(v), i), ...
            sprintf('%s%d_joint', basevariants(v), i)
        };
        for t = 1:numel(namesToTry)
            name = namesToTry{t};
            triedNames(end+1) = name; %look ahead
            [rc, handle] = sim.simxGetObjectHandle(clientID, name, sim.simx_opmode_blocking);
            fprintf('try get handle "%s" -> rc=%d\n', name, rc);
            if rc == sim.simx_return_ok
                fprintf(' found joint %d as "%s" (handle=%d)\n', i, name, handle);
                foundHandles(i) = handle;
                found = true;
                break;
            end
            pause(0.01);
        end
        if found, break; end
    end
    if ~found
        sim.simxFinish(clientID); sim.delete();
        fprintf('Tried names for joint %d:\n', i); disp(triedNames);
        error('Could not retrieve handle for joint %d. Copy exact object name from scene.', i);
    end
end
end

start streaming pose (first call starts streaming)
lm.simxGetObjectPosition(clientID, hDummy, -1, sim.simx_opmode_streaming);
lm.simxGetObjectOrientation(clientID, hDummy, -1, sim.simx_opmode_streaming);
use(0.05);

% ----- constants / limits used by IK -----
l = 0.132; k3 = 0.342; k4 = 0.372;
i = 0.2317; k6 = 0.1033; k7 = 0.00748;

lanar_link2 = k3 - k2;
lanar_link3 = sqrt((k4 - k3)^2 + k5^2);

limits_deg = [ -175, 175; % theta1
               -90, 36.7; % theta2
               -80, 90; % theta3
               -175, 175; % theta4
               -100, 110; % theta5
               -147.5, 147.5 ]; % theta6
limits_rad = deg2rad(limits_deg);

initial warm-start guess for full theta
theta_guess = deg2rad([10; 10; 10; 10; 10; 10]);

```

3) This loop runs continuously while the connection to CoppeliaSim is active. Each cycle reads the dummy's latest streamed position and orientation and, after respecting a time-limit to avoid overloading, constructs the desired end-effector transform. It then runs the IK solver to compute the best joint configuration and sends those joint targets back to the simulator. If the IK fails or takes too long, the cycle is skipped to maintain responsiveness. The loop ends only when the CoppeliaSim connection closes, after which the script cleans up and exits.

```

%% ----- main loop -----
lastSolveT = tic;
fprintf('Entering main loop - move the dummy in CoppeliaSim to update target.\n');
while sim.simxGetConnectionId(clientID) ~= -1
    % read latest dummy pose from buffer
    [rcp, pos] = sim.simxGetObjectPosition(clientID, hDummy, -1, sim.simx_opmode_buffer);
    [rco, ori] = sim.simxGetObjectOrientation(clientID, hDummy, -1, sim.simx_opmode_buffer);

    if rcp ~= sim.simx_return_ok || rco ~= sim.simx_return_ok
        % if buffer not yet available, just wait shortly
        pause(0.02);
        continue;
    end

    if printPose
        fprintf('dummy pos=[%.4f %.4f %.4f] ori=[%.4f %.4f %.4f]\n', pos(1),pos(2),pos(3), ori(1),ori(2),ori(3));
    end

    % rate-limit IK solves to avoid overload
    if toc_in_seconds(lastSolveT) < solveInterval
        pause(0.005); continue;
    end
    lastSolveT = tic_seconds();

    % build T_des using intrinsic XYZ Euler eul = [a b g]
    Rdes = eulerXYZ_to_R(ori);
    Tdes = [Rdes, pos(:); 0 0 0 1];

    % run the candidate-based IK pipeline using Tdes
    try
        t_start = tic;
        best_final = run_candidate_ik(Tdes, theta_guess, k2,k3,k4,k5,k6,k7, planar_link2, planar_link3, limits_rad);
        elapsed = toc(t_start);
        if elapsed > maxSolveTime

```

```

        if elapsed > maxSolveTime
            warning('IK solve took %.3f s (over maxSolveTime %.3f). Skipping send this cycle.', elapsed, maxSolveTime);
            continue;
        end
        if isempty(best_final) || any(isnan(best_final))
            warning('IK failed - resetting guess and skipping this cycle.');
```

$$\theta_{\text{guess}} = \text{deg2rad}([0 \ 0 \ 0 \ 0 \ 0]); \quad \% \text{ neutral pose instead of previous}$$

```

            continue;
        end

        % send best_final to CoppeliaSim (oneshot)
        for j = 1:6
            sim.simxSetJointTargetPosition(clientID, foundHandles(j), best_final(j), sim.simx_opmode_oneshot);
        end
        theta_guess = best_final(:); % warm-start next iter

        catch ME
            %warning('IK cycle failed: %s', ME.message);
            warning('IK cycle failed:');
            % keep previous guess, continue
        end

        % small pause to avoid tight loop
        pause(0.01);
    end

    % cleanup
    sim.simxFinish(clientID);
    sim.delete();
    fprintf('Connection closed, exiting.\n');
```

4) These helper functions support timing and inverse-kinematics evaluation. The first two (`tic_seconds` and `toc_in_seconds`) provide a safe way to measure elapsed time, returning infinity if the input timer is invalid; this is used to rate-limit how often IK is computed. The main function shown, `run_candidate_ik`, receives the target transform of the dummy and generates all feasible IK candidates for the first three joints by computing wrist position, exploring both shoulder configurations, and checking geometric solutions for the elbow. For each candidate it evaluates forward kinematics, measures position and orientation error against the desired pose, and stores the results with a weighted error score. These candidates are later sorted so the full IK pipeline can select the best-matching joint configuration.

```

%% ----- Utility & IK pipeline functions -----

function sec = tic_seconds()
    sec = tic;
end
function elapsed = toc_in_seconds(tic_val)
    % if tic_val is negative or empty -> large number
    if isempty(tic_val) || ~isnumeric(tic_val)
        elapsed = inf;
        return;
    end
    % interpret tic_val as tic() token; return elapsed seconds
    elapsed = toc(tic_val);
end

function best_final = run_candidate_ik(T_des, theta_guess, k2,k3,k4,k5,k6,k7, planar_link2, planar_link3, limits_rad)
    % This is the merged IK pipeline from your code, but using T_des (from dummy).
    % Returns best_final as 1x6 vector of joint angles (radians) or [] if none found.

    % Setup
    R_des = T_des(1:3,1:3);
    p_des = T_des(1:3,4);

    % compute wrist global target from T_des (replace previous Ttotal usage)
    wrist_global_x = -k6 * T_des(1,2) - k7 * T_des(1,3) + T_des(1,4);
    wrist_global_y = -k6 * T_des(2,2) - k7 * T_des(2,3) + T_des(2,4);
    wrist_global_z = -k6 * T_des(3,2) - k7 * T_des(3,3) + T_des(3,4);

    global_x = -0.100; global_y = -0.631; global_z = 0.051;
    px = wrist_global_x - global_x;
    py = wrist_global_y - global_y;
    pz = wrist_global_z - global_z;

    % theta1 candidates

    % theta1 candidates
    theta1_a = atan2(-px, py);
    theta1_b = wrapToPi(theta1_a + pi);
    theta1_list = [theta1_a, theta1_b];

    % wrist initial guess (from theta_guess)
    t4_guess = theta_guess(4);
    t5_guess = theta_guess(5);
    t6_guess = theta_guess(6);

    candidates = []; % columns: theta1..theta6 | pos_err | ori_err | total_err

    for idx = 1:2
        theta1 = theta1_list(idx);

        % rotate wrist coordinates into joint-2 frame
        px_dot = px * cos(theta1) + py * sin(theta1);
        py_dot = -px * sin(theta1) + py * cos(theta1);
        pz_dot = pz - k2;

        % compute theta3 and theta2 candidates
        [theta3_1, theta3_2, theta2_22, theta2_44] = compute_theta3(px_dot, py_dot, pz_dot, planar_link2, planar_link3);

        theta3_options = [theta3_1, theta3_2];
        theta2_options = [theta2_44, theta2_22];

        for j = 1:2
            t3_geom = theta3_options(j);
            t2_geom = theta2_options(j);
            if isempty(t2_geom) || isempty(t3_geom), continue; end

            if j == 1
                t3_deg = 80 + rad2deg(t3_geom);
                t2_deg = 90 - rad2deg(t2_geom);
            end
        end
    end
end

```

```

else
    t3_deg = 80 - rad2deg(t3_geom);
    t2_deg = -90 + rad2deg(t2_geom);
end

t1_candidate = theta1;
t2_candidate = deg2rad(t2_deg);
t3_candidate = deg2rad(t3_deg);

theta_candidate = [t1_candidate, t2_candidate, t3_candidate, t4_guess, t5_guess, t6_guess];

% joint limits
in_limits = all(theta_candidate >= (limits_rad(:,1)'-1e-9) & theta_candidate <= (limits_rad(:,2)'+1e-9));
if ~in_limits, continue; end

[T_cand, ~, ~, ~] = compute_finalA(theta_candidate);

Re = eye(3);
pe = [0; k5 + k6; k4 + k7];
Te = [Re, pe; 0 0 0 1];
T_cand_total = T_cand * Te;

p_c = T_cand_total(1:3,4);
pos_err = norm(p_c - p_des);

R_c = T_cand_total(1:3,1:3);
ori_err = angleBetweenRot(R_c, R_des);

w_pos = 1.0; w_ori = 0.5;
total_err = w_pos * pos_err + w_ori * ori_err;

candidates = [candidates; theta_candidate, pos_err, ori_err, total_err]; %#ok<AGROW>
end
end

```

5) This part takes all the geometric IK candidates computed so far and selects the best overall solution. First, if no shoulder–elbow candidates were found, it immediately returns failure. Otherwise, it sorts those candidates by their total error and picks the best one as the “base” solution for joints 1–3. Using that base, it computes all eight possible wrist joint combinations (because each of joints 4, 5, and 6 has two valid analytic solutions). Each full 6-joint candidate is then evaluated with forward kinematics, checked against joint limits, and scored by position and orientation error. Finally, the script sorts these wrist-augmented candidates and returns the single best valid configuration as the final IK result.

```

if isempty(candidates)
    best_final = [];
    return;
end

[~, order] = sort(candidates(:, end));
candidates_sorted = candidates(order, :);
best = candidates_sorted(1,:);
best_theta = best(1:6);

% compute wrist joint possibilities for that best shoulder/elbow (use T_des)
shoulder_elbow_angles = [best_theta(1), best_theta(2), best_theta(3)];
[t4_1,t4_2, t5_1, t5_2, t6_1, t6_2] = compute_wrist_joints(shoulder_elbow_angles, T_des);

final_candidates = [
    best_theta(1), best_theta(2), best_theta(3), t4_1, t5_1, t6_1;
    best_theta(1), best_theta(2), best_theta(3), t4_1, t5_1, t6_2;
    best_theta(1), best_theta(2), best_theta(3), t4_1, t5_2, t6_1;
    best_theta(1), best_theta(2), best_theta(3), t4_1, t5_2, t6_2;
    best_theta(1), best_theta(2), best_theta(3), t4_2, t5_1, t6_1;
    best_theta(1), best_theta(2), best_theta(3), t4_2, t5_1, t6_2;
    best_theta(1), best_theta(2), best_theta(3), t4_2, t5_2, t6_1;
    best_theta(1), best_theta(2), best_theta(3), t4_2, t5_2, t6_2;
];

% evaluate final candidates
results = zeros(size(final_candidates,1), 4);
for i = 1:size(final_candidates,1)
    cand = final_candidates(i,:);
    if any(cand < (limits_rad(:,1)'-1e-9)) || any(cand > (limits_rad(:,2)'+1e-9))
        pos_err = Inf; ori_err = Inf; total_err = Inf;

    else
        [T_cand, ~, ~, ~] = compute_finalA(cand);
        Re = eye(3);
        pe = [0; k5 + k6; k4 + k7];
        Te = [Re, pe; 0 0 0 1];
        T_cand_total = T_cand * Te;

        p_c = T_cand_total(1:3,4);
        pos_err = norm(p_c - p_des);
        R_c = T_cand_total(1:3,1:3);
        ori_err = angleBetweenRot(R_c, R_des);
        total_err = 1.0 * pos_err + 0.5 * ori_err;
    end
    results(i,:) = [pos_err, ori_err, total_err, i];
end

[~, ord_final] = sort(results(:,3));
final_candidates_sorted = final_candidates(ord_final, :);

% pick best valid candidate
if isempty(final_candidates_sorted)
    best_final = [];
else
    best_final = final_candidates_sorted(1,:);
end
end

```

6)

`wrapToPi` simply takes any angle and maps it into the interval $[-\pi, \pi]$, ensuring consistent angle wrapping for IK computations. The function `compute_finalA` performs the robot's forward kinematics: it extracts the six joint angles, computes their sines and cosines, and builds the homogeneous transformation matrices A_{0A_0A0} through A_{6A_6A6} that represent each joint and link of the Niryo robot. By multiplying all these matrices, it obtains T_{final} , the transformation of the wrist frame in the world coordinate system, and also computes T_{total} , the full end-effector pose including the tool offset. It returns both the Cartesian position of the wrist and the transformed wrist point x , which is later used in the IK solver for accuracy evaluation.

```
%% --- utility local functions (same as in your original script) ---
function y = wrapToPi(x)
    y = mod(x + pi, 2*pi) - pi;
end

function [Tfinal, Ttotal, pos, x] = compute_finalA(theta)
    t1 = theta(1); t2 = theta(2); t3 = theta(3);
    t4 = theta(4); t5 = theta(5); t6 = theta(6);

    c1 = cos(t1); s1 = sin(t1);
    c2 = cos(t2); s2 = sin(t2);
    c3 = cos(t3); s3 = sin(t3);
    c4 = cos(t4); s4 = sin(t4);
    c5 = cos(t5); s5 = sin(t5);
    c6 = cos(t6); s6 = sin(t6);

    k2 = 0.132; k3 = 0.342; k4 = 0.372; k5 = 0.2317; k6 = 0.1033; k7 = 0.00748;

    A0 = [ 1, 0, 0, -0.100;
           0, 1, 0, -0.631;
           0, 0, 1, 0.051;
           0, 0, 0, 1 ];

    A1 = [ c1, -s1, 0, 0;
           s1, c1, 0, 0;
           0, 0, 1, 0;
           0, 0, 0, 1 ];

    A2 = [ 1, 0, 0, 0;
           0, c2, -s2, k2*s2;
           0, s2, c2, k2*(1 - c2);
           0, 0, 0, 1 ];
```

```

A3 = [ 1, 0, 0, 0;
       0, c3, -s3, k3*s3;
       0, s3, c3, k3*(1 - c3);
       0, 0, 0, 1 ];

A4 = [ c4, 0, s4, -k4*s4;
       0, 1, 0, 0;
       -s4, 0, c4, k4*(1 - c4);
       0, 0, 0, 1 ];

A5 = [ 1, 0, 0, 0;
       0, c5, -s5, k5*(1 - c5) + k4*s5;
       0, s5, c5, -k5*s5 + k4*(1 - c5);
       0, 0, 0, 1 ];

A6 = [ c6, 0, s6, -k4*s6;
       0, 1, 0, 0;
       -s6, 0, c6, k4*(1 - c6);
       0, 0, 0, 1 ];

Tfinal = A0 * A1 * A2 * A3 * A4 * A5 * A6;
pos = Tfinal*[0;0;0;1];

Re = eye(3);
pe = [0; k5 + k6; k4 + k7];
Te = [Re, pe; 0 0 0 1];
Ttotal = Tfinal * Te;

wrist_transformation = A4 * A5 * A6;
wrist_position = [0; k5; k4; 1];
x = wrist_transformation * wrist_position;
end

```

7)

`compute_theta3` solves the geometric IK for joints 2 and 3: it applies the law of cosines using the projected wrist position to compute two possible elbow angles (θ_3) and, for each, computes the corresponding shoulder angle (θ_2).

`angleBetweenRot` measures orientation error between two rotation matrices by converting their relative rotation into a single angular distance.

`compute_wrist_joints` takes the solved shoulder–elbow configuration, isolates the rotation that the wrist must produce, and analytically derives all valid combinations for θ_4 , θ_5 , and θ_6 by exploiting the structure of the rotation matrix and handling singular cases when θ_5 approaches zero. Finally, `eulerXYZ_to_R` builds a rotation matrix from XYZ intrinsic Euler angles so the IK solver can work

with the dummy's orientation in standard matrix form.

```
function [theta3_1, theta3_2, theta2_22, theta2_44] = compute_theta3(px_dot, py_dot, pz_dot, planar_link2, planar_link3)
    numerator = px_dot.^2 + py_dot.^2 + pz_dot.^2 - planar_link2.^2 - planar_link3.^2;
    denominator = 2 * planar_link2 .* planar_link3;
    D = numerator ./ denominator;
    D_clamped = min(max(D, -1), 1);
    D_sine = sqrt( max(0, 1 - D_clamped.^2) );

    theta3_1 = atan2(-D_sine, D_clamped);
    theta3_2 = atan2( D_sine, D_clamped);

    r = sqrt(px_dot.^2 + py_dot.^2);
    theta2_22 = atan2(pz_dot, r) - atan2(planar_link3 .* sin(theta3_1), planar_link2 + planar_link3 .* cos(theta3_1));
    theta2_44 = atan2(pz_dot, r) - atan2(planar_link3 .* sin(theta3_2), planar_link2 + planar_link3 .* cos(theta3_2));
end

function ang = angleBetweenRot(R1, R2)
    Rerr = R1' * R2;
    tr = trace(Rerr);
    cosang = (tr - 1)/2;
    cosang = min(max(cosang, -1), 1);
    ang = acos(cosang);
end

function [theta4_1, theta4_2, theta5_1, theta5_2, theta6_1, theta6_2] = compute_wrist_joints(thetas, Tdes)
    k2 = 0.132; k3 = 0.342;
    t1 = thetas(1); t2 = thetas(2); t3 = thetas(3);
    c1 = cos(t1); s1 = sin(t1);
    c2 = cos(t2); s2 = sin(t2);
    c3 = cos(t3); s3 = sin(t3);

    A0 = [ 1, 0, 0, -0.100;
           0, 1, 0, -0.631;
           0, 0, 1, 0.051;
           0, 0, 0, 1 ];
    A1 = [ c1, -s1, 0, 0;
           s1,  c1, 0, 0;
           0,  0, 1, 0;
           0,  0, 0, 1 ];
```

```

A2 = [ 1, 0, 0, 0;
       0, c2, -s2, k2*s2;
       0, s2, c2, k2*(1 - c2);
       0, 0, 0, 1 ];
A3 = [ 1, 0, 0, 0;
       0, c3, -s3, k3*s3;
       0, s3, c3, k3*(1 - c3);
       0, 0, 0, 1 ];

T034 = A3 \ (A2 \ (A1 \ (A0 \ Tdes) ));
R = T034(1:3,1:3);

sy = sqrt(R(1,2)^2 + R(3,2)^2);
theta5_1 = atan2(sy, R(2,2));
theta5_2 = atan2(-sy, R(2,2));

if abs(sin(theta5_1)) > 1e-9
    theta4_1 = pi - atan2(-R(1,2)/sin(theta5_1), R(3,2)/sin(theta5_1));
    theta6_1 = atan2(R(2,1)/sin(theta5_1), -R(2,3)/sin(theta5_1));
else
    theta4_1 = 0;
    theta6_1 = atan2(-R(1,3), R(1,1));
end

if abs(sin(theta5_2)) > 1e-9
    theta4_2 = pi - atan2(-R(1,2)/sin(theta5_2), R(3,2)/sin(theta5_2));
    theta6_2 = atan2(R(2,1)/sin(theta5_2), -R(2,3)/sin(theta5_2));
else
    theta4_2 = 0;
    theta6_2 = atan2(-R(1,3), R(1,1));
end
end

function R = eulerXYZ_to_R(eul)
    a = eul(1); b = eul(2); g = eul(3);
    Rx = @(x)[1 0 0; 0 cos(x) -sin(x); 0 sin(x) cos(x)];
    Ry = @(y)[cos(y) 0 sin(y); 0 1 0; -sin(y) 0 cos(y)];
    Rz = @(z)[cos(z) -sin(z) 0; sin(z) cos(z) 0; 0 0 1];
    R = Rz(g) * Ry(b) * Rx(a);
end

```